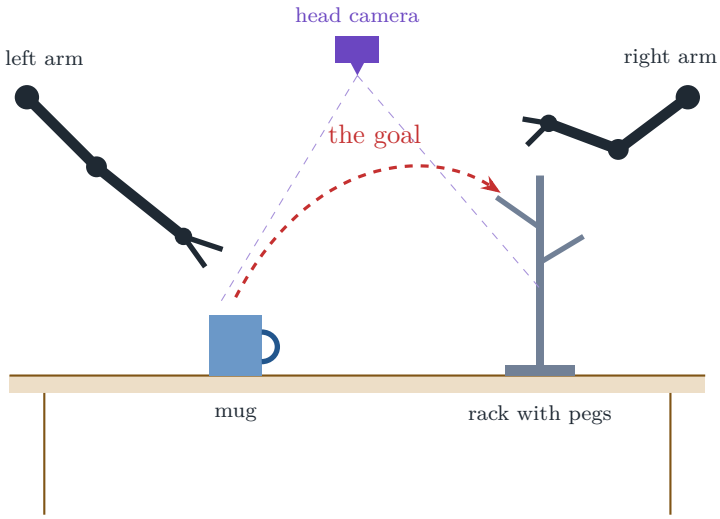
How Our Robot Learns to Hang a Mug

A friendly guide to the Agent-Aware VLA harness

Version 1 • October 2026



This guide explains the project from zero: what the robot does, who the "team" inside the software is, how one episode runs, how the code is organised, how to change things, and how to set it up and run it. Coloured boxes help you find your way:

simple explanation | key idea | where to change it | careful

Covers branch `refactor`. File and line references point to that code.

Contents

1 The story in one page **2**

2 Words you will see **3**

3 The world the robot lives in **4**
3.1 The scene and the three cameras . . . 4
3.2 Fourteen numbers drive the robot . . . 4
3.3 Two kinds of time . . . 4

4 The task is split into three phases **5**

5 One turn of the loop **5**
5.1 What happens after a skill finishes? . . . 6
5.2 Budgets: how much the robot may try . . . 6

6 The six move cards (skills) **7**
6.1 How a VLA call plays a chunk . . . 7
6.2 How a careful (analytic) move works . . . 8

7 House rules that keep things safe **8**
7.1 Three answers, not two . . . 8
7.2 The wall between public and private knowledge . . . 8
7.3 Opening a hand needs proof . . . 8
7.4 Other rules in one list . . . 9

8 The eyes: perception **9**
8.1 From a pixel to a 3D point . . . 9
8.2 Who looks at the pictures? . . . 10
8.3 Three points make a frame . . . 11
8.4 Is the mug lifted? Measuring against the table . . . 11
8.5 When a chosen pixel is bad . . . 12

9 Architecture: what runs where **12**
9.1 How the code layers depend on each other . . . 12

10 Where everything lives **13**
10.1 Which file do I open? . . . 13

11 Code tour: how it works today **14**
11.1 Starting an episode . . . 14
11.2 Choosing the coach . . . 14
11.3 What a move card looks like . . . 15
11.4 The heart of the turn loop . . . 15
11.5 Playing a policy chunk . . . 15

12 Cookbook: I want to. . . **16**

13 Configuration files **17**

14 Setting up and running **18**
14.1 What you get after a run . . . 19
14.2 Other ways to run . . . 19

15 Where things stand today **20**

16 Testing and contributing **20**

A Frequently asked questions **21**

1 The story in one page

Like I'm five

There is a robot with **two arms** standing at a table. On the table there is a **mug**. Next to it stands a little tree-shaped **rack** with pegs sticking out. The robot's job is to pick up the mug and **hang it on a peg by its handle**.
The robot already learned how to do this by *watching lots of examples*, a bit like a child copying a grown-up. That learned skill is called the **policy**. It is good at most of the job but often gets wobbly at the very end, when the handle has to slide onto the peg.
We are **not** allowed to teach the robot again. Instead, we add a **coach** who watches through the robot's cameras every few moments and says "keep going", "try this careful move instead", or "stop, this is not safe". The coach can only choose from **six move cards**. It never wiggles the robot's joints by itself.

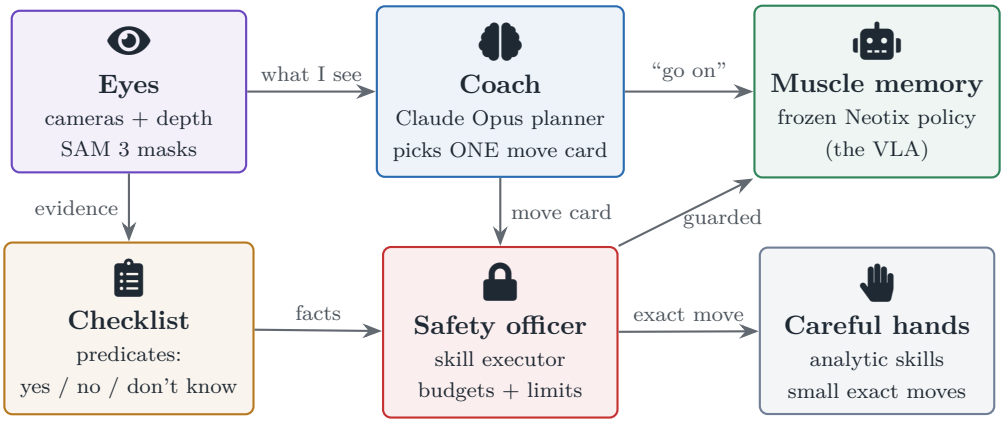


Figure 1: The "team" inside the software. Each box is a real part of the code.

Key idea

One move card per turn. The coach (Opus) reads the camera pictures and a few facts, then picks exactly one bounded skill. The software checks the card, the robot does it, and the coach gets fresh pictures. Nothing the coach says can move a joint directly.

Why is it built like this?

- **The policy is frozen.** It was trained elsewhere and is published as `Neotix-Robotics/robotwin-hanging-mug-dit`. We only *use* it.
- **Language models are bad at joint angles** but good at looking at pictures and judging "is this going well?". So Opus judges and chooses, and code does the exact maths.
- **Safety first.** Every move has hard limits (how far, how fast, how many tries). If anything is uncertain, the robot stops instead of guessing.

2 Words you will see

Word	Plain meaning
Joint	A robot "elbow" that can bend. Each arm has six.
Gripper	The robot's two-finger hand. `0` = closed, `1` = open.
TCP / end effector	The point between the fingers. Moves are described by where this point goes.
14-DoF	14 numbers that control both arms: 6 joints + 1 gripper per arm.
Policy / VLA	"Vision-Language-Action" model: pictures in, robot actions out. Ours is *frozen*.
Chunk	The policy answers with a short list of actions (e.g. 8) at once.
Simulator	A video-game-like world (RoboTwin) with real physics, so nothing breaks.
Episode / seed	One attempt at the task. The seed decides where the mug starts.
Observation	One snapshot: 3 camera pictures + the 14 numbers. Each has a sequence number.
RGB-D	Colour picture (RGB) plus Depth: how far away each pixel is.
Calibration	The camera's "lens settings" (intrinsic) and position (extrinsic). Needed to turn pixels into 3D points.
Mask / SAM 3	A cut-out showing which pixels are the mug, rack or table. SAM 3 makes them.
Planner	The coach. Either Claude Opus or a fixed script.
Skill	One of the six move cards (Section 6).
Predicate	A checklist question like "is the mug lifted?" Answer: `true`, `false` or `null` (unknown).
Anchor	A named 3D target such as `mug_pregrasp` or `rack_seat`, freshly computed each time.
Oracle	"Cheating" simulator knowledge (exact mug position). Only for debugging, never shown to the coach.
Modal	A cloud service that runs our GPU models (the policy and SAM 3).

3 The world the robot lives in

3.1 The scene and the three cameras

The simulator is **RoboTwin** running the `hanging_mug` task. The robot has three cameras. Every observation contains all three pictures.

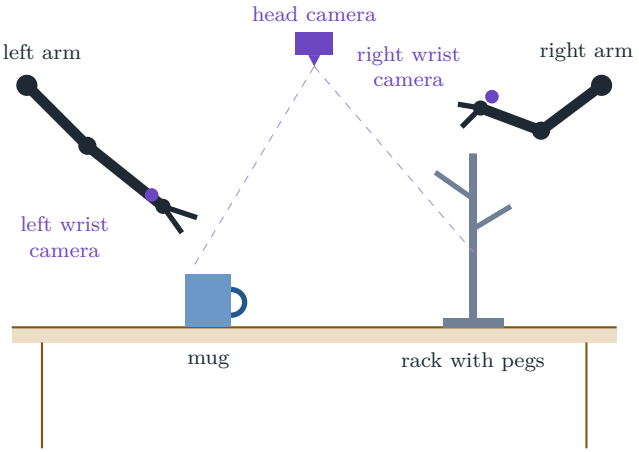


Figure 2: The scene. Camera names in the code: `head_camera`, `left_camera`, `right_camera`.

3.2 Fourteen numbers drive the robot

The policy reads the pictures plus 14 numbers, and answers with 14 new numbers per step.

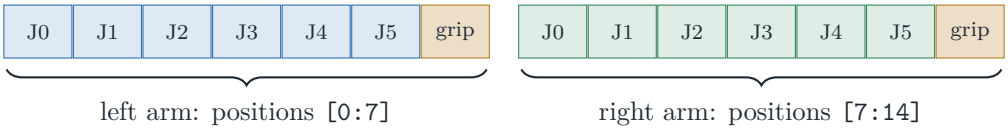


Figure 3: The 14-number action layout, defined once in `harness/environment/layout.py`.

3.3 Two kinds of time

- **Policy actions**: one action = one call to RoboTwin's `take_action`, which itself runs many physics steps. The policy sends actions in chunks.
- **Physics ticks**: the careful analytic skills move the arm one tick at a time. Physics runs at **250 ticks per second**, so 125 ticks is half a second.

4 The task is split into three phases

Like I'm five

Hanging a mug is hard to do in one go, so the robot does it in three chapters. The left hand picks up the mug. Then it puts it back down nearby so the right hand can take it. Then the right hand picks it up and hangs it on the rack. You can only go forward to the next chapter, never back.

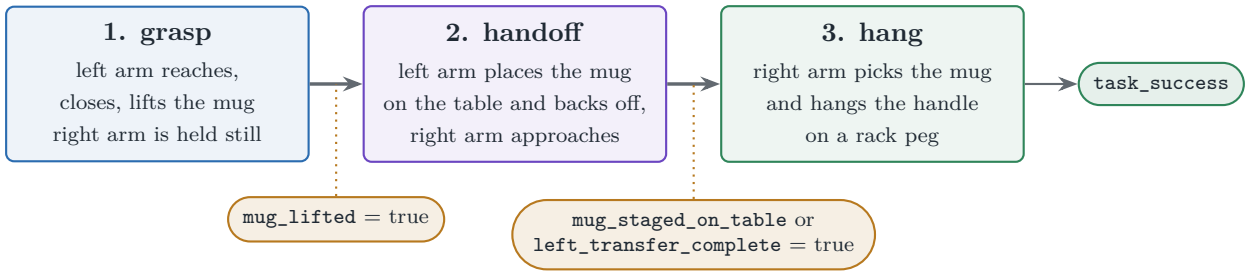


Figure 4: Phases and the public milestones that unlock the next phase.

Key idea

A **true public milestone** sets the minimum phase. When the cameras cannot tell (the answer is `null`), Opus may move the phase forward by its own visual judgement, but only **one step per turn**. The hang phase additionally needs the staged-mug milestone. See </> harness/planner/anthropic.py:294 (`_public_phase_floor`) and `_resolve_phase`.

5 One turn of the loop

Every episode is a loop of **turns**. One turn = look, decide, check, do, record.

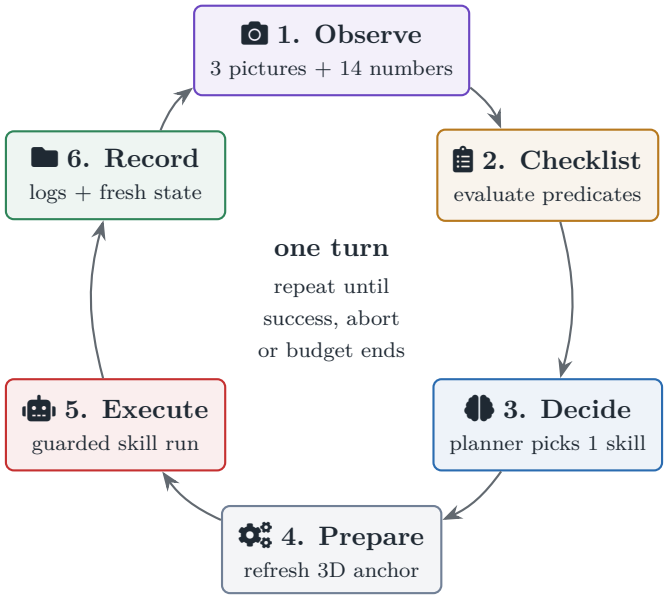


Figure 5: The turn loop, implemented in `SkillOrchestrator.run` (</> harness/runtime/orchestrator.py:219).

5.1 What happens after a skill finishes?

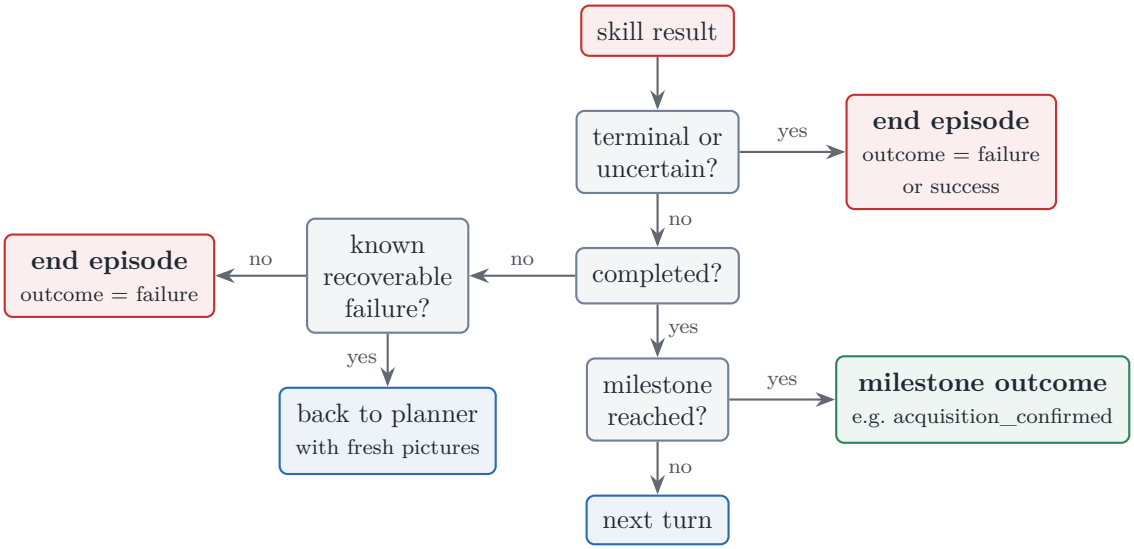


Figure 6: Turn outcome decisions. "Recoverable" failures are a short, explicit list in `recovery_flags` (</> `harness/runtime/orchestrator.py:143`).

The recoverable failures are: a move that was rejected *before* moving, a move that moved but missed its target, a contact between the two arms, and a temporary perception gap. In every case the planner sees fresh pictures; **motion is never replayed automatically**.

5.2 Budgets: how much the robot may try

Budget (`EpisodeBudget`)	Default	Meaning
`max_skill_calls`	100	move cards per episode
`max_policy_actions`	1200	policy actions executed in total
`max_vla_chunks`	150	times the policy may be asked
`max_analytic_ticks`	12000	physics ticks for careful moves (48 s)
`max_turns` (`RunSettings`)	20	planner turns

</> `harness/skills/schemas.py:123`. Configs usually override these.

6 The six move cards (skills)

Skill	What it does	Arms	Limits
`vla_act`	Ask the frozen policy for short chunks and play them, with checks after every action.	both	≤32 chunks, ≤24 actions/chunk
`move_to`	Move one hand to a fresh named target (or current pose) plus a small offset.	one	offset ≤ 20 cm
`rotate_wrist`	Turn one wrist around the vertical axis.	one	≤ 30°
`set_gripper`	Slowly open/close one hand. Opening needs permission.	one	50–500 ticks ramp
`release`	Fully open one hand, only when support is proven.	one	guarded
`abort`	Stop the episode. No motion.	none	–

</> `harness/skills/schemas.py:13`. Blue skills are *analytic*: exact maths, no learning.

6.1 How a VLA call plays a chunk

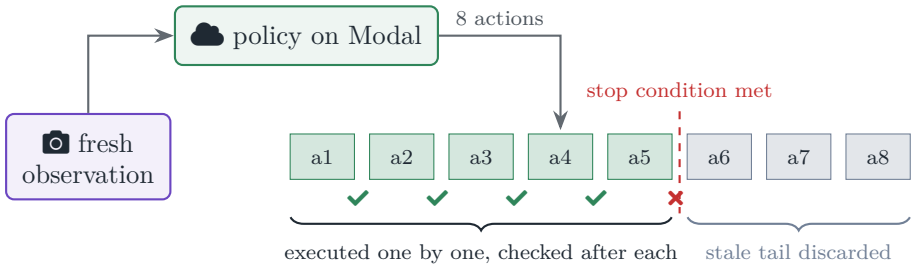


Figure 7: One chunk. After every action the code checks the stop predicate, arm-to-arm contact and the task checker. Unused actions are thrown away, never executed later (</> `harness/skills/vla_act.py:159`).

Key idea

Feedback boundary. In Opus mode each `vla_act` call ends after `planner_feedback_chunks` chunks (normally 1). The coach then sees new pictures and decides again. When RoboTwin's checker first reports success, the policy may finish a fixed 16-action follow-through so the gripper can let go cleanly (`NATIVE_SUCCESS_FOLLOW_THROUGH_ACTIONS`).

6.2 How a careful (analytic) move works

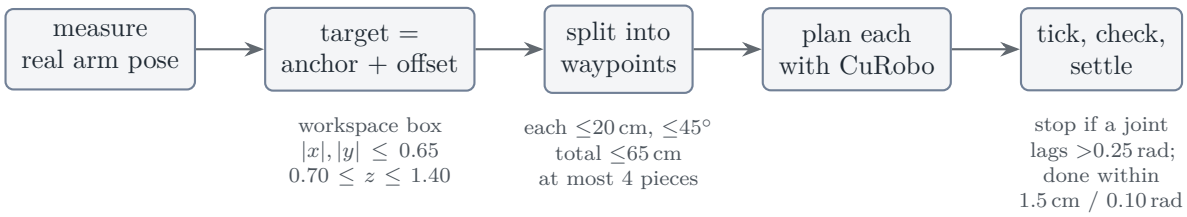


Figure 8: Analytic skills always start from the *measured* pose, never a remembered one (</> `harness/skills/analytic.py`).

7 House rules that keep things safe

7.1 Three answers, not two

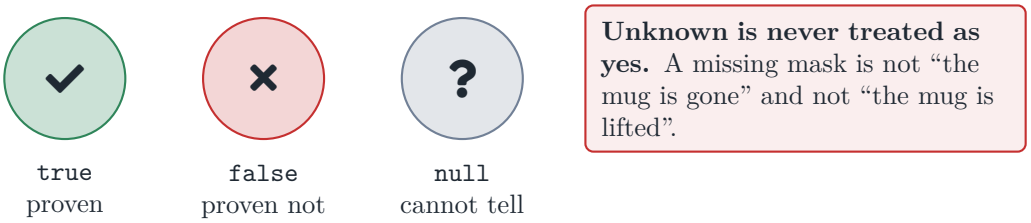


Figure 9: Predicate answers (`PredicateResult`, </> `harness/predicates/base.py`).

7.2 The wall between public and private knowledge

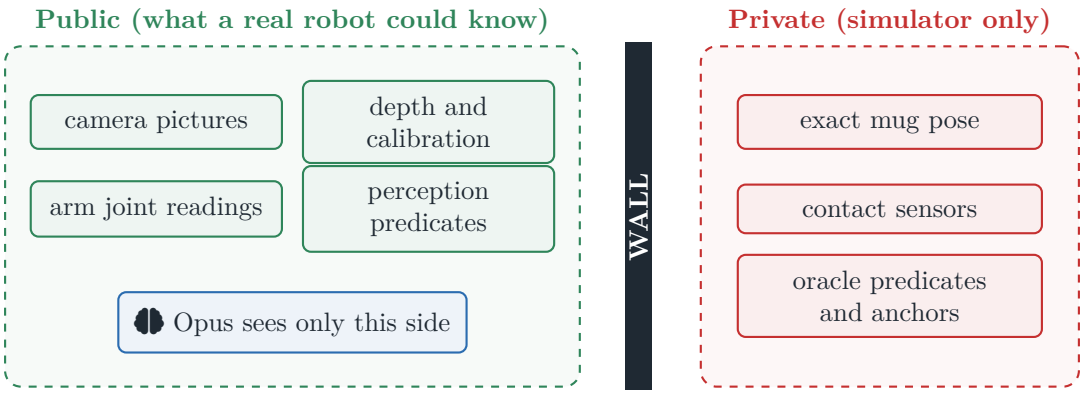


Figure 10: The oracle may gate execution during debugging (`--development-oracle`), but it is filtered out before anything is sent to Opus (</> `harness/planner/prompt_builder.py`).

7.3 Opening a hand needs proof

The robot may only open a hand that is holding the mug when the public checklist says that `release_ready` is `true` (the mug is resting on the table), or, at the rack, that `rack_seat_aligned` is `true`. The planner enforces this before anything moves:

Listing 1: harness/planner/anthropic.py:181

```
release_authorized = self._public_predicate_true(context, "release_ready")
rack_seat_authorized = (vla_phase == "hang" and
    self._public_predicate_true(context, "rack_seat_aligned"))
release_allowed = (release_authorized or
    (command.action.value == "release"
     and command.parameters.authorization == "rack_seat_alignment"
     and rack_seat_authorized))
if (not release_allowed and
        (command.action.value == "release"
         or (command.action.value == "set_gripper" and command.parameters.value > 0.0))):
    raise InvalidPlannerResponse("release_not_publicly_authorized")
```

7.4 Other rules in one list

- **Never retry an uncertain command.** If a response is lost after sending an action, the episode stops.
- **Anchors expire.** A target is valid only for the observation it was computed from.
- **Arm contact recovery is narrow**: right hand moves 3–10 cm away, at most 2 retreats and 3 incidents.
- **Grounding retries are limited**: a rejected pixel can be re-chosen at most twice, the second time from a different camera.
- **Logs never contain secrets** or raw provider text; only short error codes.

8 The eyes: perception

Like I'm five

A picture is flat. To reach for the mug, the robot needs to know *where in 3D* it is. Each camera also tells how far away every pixel is (depth). If you know which pixel is the mug and how far away that pixel is, and where the camera is, you can work out the mug's 3D position.

8.1 From a pixel to a 3D point

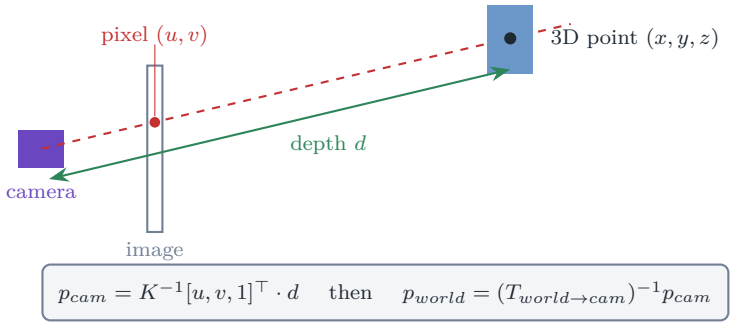


Figure 11: Unprojection. K is the intrinsic matrix, T the extrinsic pose.

Listing 2: harness/perception/rgbd.py:97 `RGBDCamera.unproject` (abridged)

```
depth = self.sample_depth(u, v, radius=radius)      # local median, ignores zeros
pixel = np.array([float(u), float(v), 1.0])
camera_xyz = np.linalg.solve(self.intrinsic, pixel) * depth
camera_h = np.append(camera_xyz, 1.0)
world = np.linalg.solve(self.world_to_camera, camera_h)
return tuple(float(value) for value in world[:3] / world[3])
```

8.2 Who looks at the pictures?

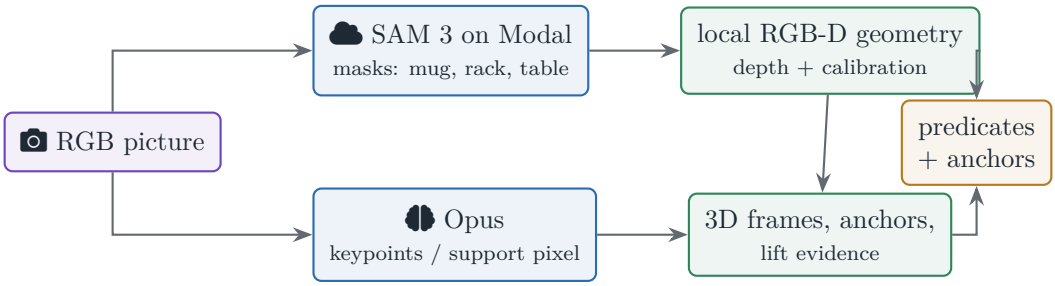


Figure 12: Only RGB images leave the machine. Depth maths always stays local.

- **SAM 3** (`harness/perception/modal_sam3.py`) draws masks. Mask confidence floors: mug 0.40, rack 0.60 (0.18 with the exploration prompt "mug rack"), table 0.60.
- **Opus keypoints** (`opus_frame.py`) give three ordered points on the rack peg or the mug handle; three points define a little 3D coordinate frame.
- **Opus support pixel** (`table_plane.py`) marks the bottom of the mug, used to measure lift height.

8.3 Three points make a frame

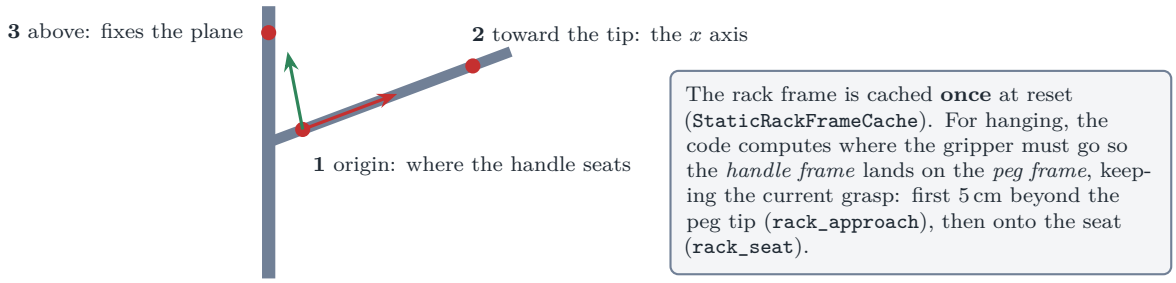


Figure 13: Keypoints become frames (</> harness/perception/pose_grounding.py, hang_targets.py).

8.4 Is the mug lifted? Measuring against the table

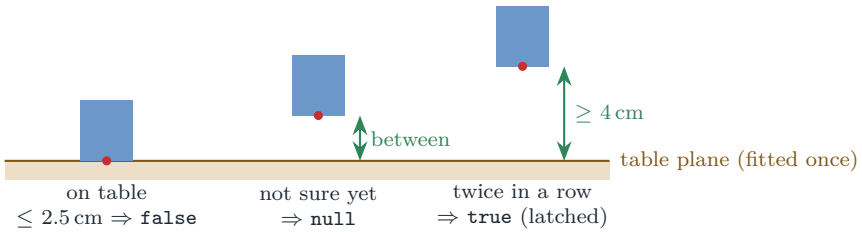


Figure 14: Table-relative lift evidence (MugLiftEvidenceTracker, </> harness/perception/table_plane.py:196).

Where to change it

Lift thresholds are constructor arguments: `lifted_distance_m=0.04`, `table_distance_m=0.025`, `consecutive_observations=2` in `harness/perception/table_plane.py`. Mask confidence floors are `MUG_MASK_MIN_CONFIDENCE` and `EXPLORE_RACK_MASK_MIN_CONFIDENCE` in `harness/runtime/assembly.py`. Prompts for Opus perception live in `harness/prompts/`.

8.5 When a chosen pixel is bad

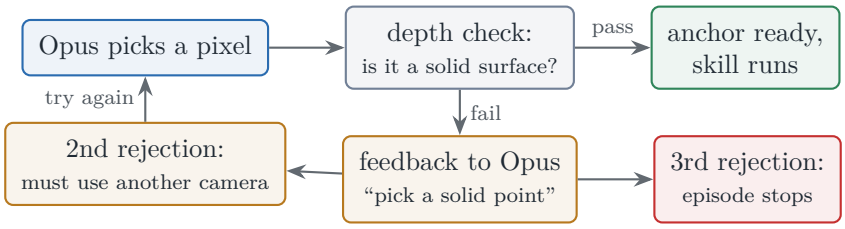


Figure 15: Grounding reselection (`MAX_GROUNDING_RESELECTIONS = 2`, `harness/runtime/orchestrator.py:19`).

9 Architecture: what runs where

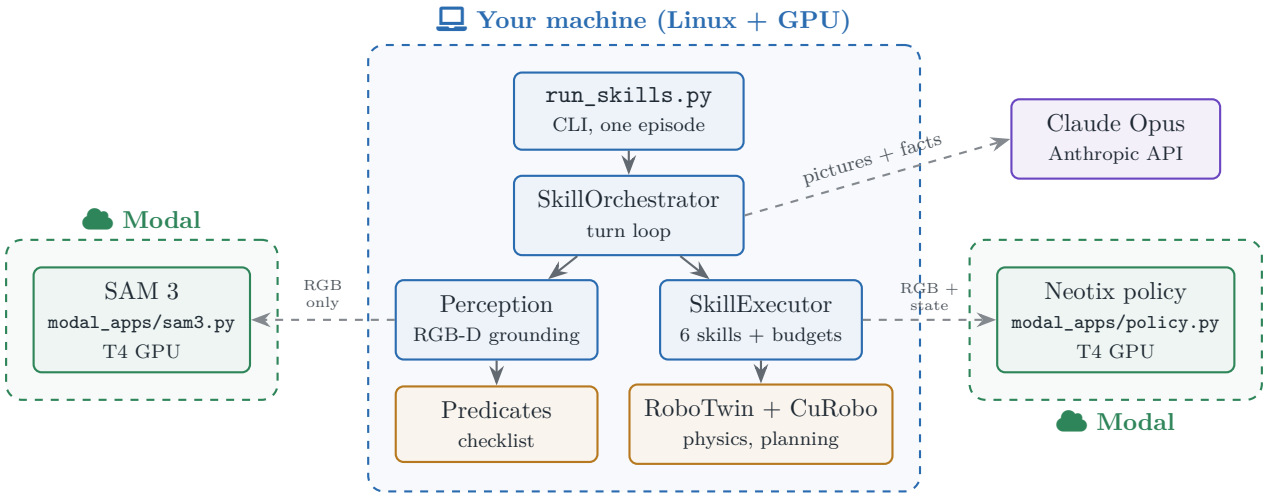


Figure 16: System architecture. Dashed arrows cross the network.

9.1 How the code layers depend on each other

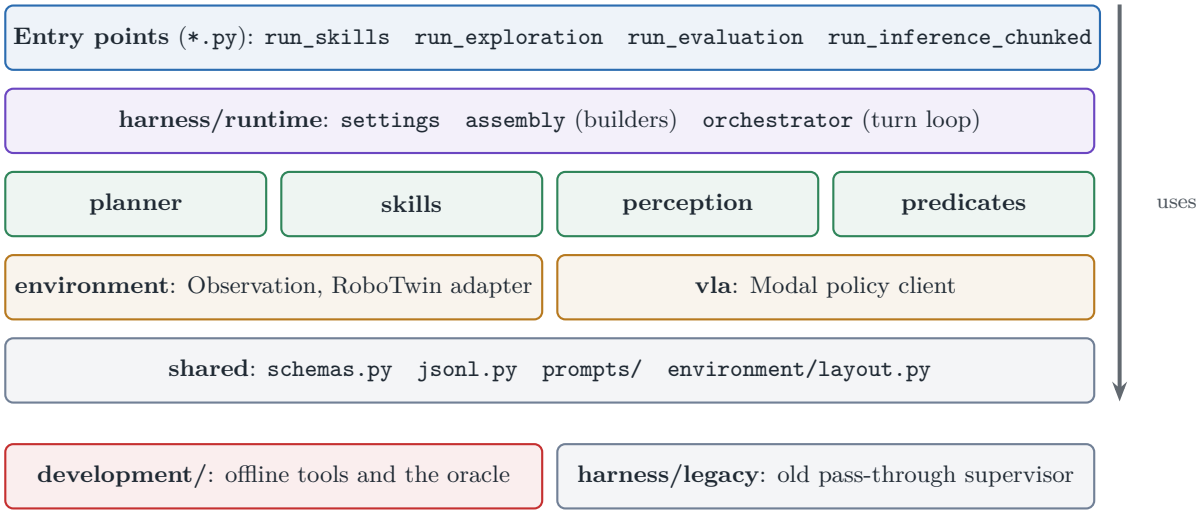


Figure 17: Higher layers use lower layers, never the other way round. `development/` is used only by the CLIs and tests.

10 Where everything lives

```
Agent-Aware-Vision-Language-Action-Model/
├─ run_skills.py                one supervised episode
├─ run_exploration.py           many episodes, one process each
├─ run_evaluation.py            frozen, fair evaluation
├─ run_inference_chunked.py     baseline: policy alone
├─ harness/
│   ├─ runtime/                 settings, builders, turn loop
│   ├─ planner/                 Opus + scripted coaches, prompt, parser
│   ├─ skills/                  the six move cards
│   ├─ perception/              eyes: masks, depth, frames
│   ├─ predicates/              the checklist
│   ├─ environment/             observation, RoboTwin adapter
│   ├─ vla/                     talk to the policy on Modal
│   ├─ memory/  exploration/  evaluation/
│   ├─ prompts/                 text sent to Opus
│   └─ legacy/                  old pass-through supervisor
├─ modal_apps/                  policy.py, sam3.py (cloud)
├─ development/
│   ├─ simulator/  replays/  capture/
│   └─ checkpoints/  recovery/  demos/
├─ configs/                     YAML presets
└─ requirements/  tests/  patches/
```

Figure 18: Repository layout. `RoboTwin/`, `data/` and `docs/` are local and not in git.

10.1 Which file do I open?

I want to understand...	Open
how an episode is started and cleaned up	`run_skills.py`
how the parts are built and connected	`harness/runtime/assembly.py`
the turn-by-turn loop	`harness/runtime/orchestrator.py`
how Opus is asked and checked	`harness/planner/anthropic.py`
what Opus is told	`harness/prompts/common.md`, `full_task.md`
what a move card may contain	`harness/skills/schemas.py`
how the policy chunk is played	`harness/skills/vla_act.py`
how careful moves are executed	`harness/skills/analytic.py`
how pictures become 3D anchors	`harness/perception/grounding.py`, `preparation.py`
the checklist logic	`harness/predicates/`
how RoboTwin is started	`harness/environment/robotwin.py`
the cloud model code	`modal_apps/`

11 Code tour: how it works today

11.1 Starting an episode

`main()` checks the command-line options *before* anything expensive starts, then `run_episode()` owns the simulator for exactly one reset:

Listing 3: `run_skills.py:191` `run_episode` (abridged)

```
planner = assembly.build_planner(args.planner, settings, planner_settings=planner_settings,
                                 pixel_grounding=args.planner_pixel_grounding)
simulator = LocalSimulator(args.robotwin_root.resolve(), rgbd_enabled=...)
environment = open_environment(simulator, video=video, frozen=frozen)
environment.reset(args.seed)  # Exactly one reset, owned by this runner.

if settings.perception.opus_rack_grounding:          # optional: ground the rack once
    rack_frame_cache, rack_grounding = assembly.initialize_rack_frame(...)
if settings.perception.opus_mug_lift_grounding:      # optional: table-plane lift evidence
    assembly.initialize_mug_lift_evidence(...)
predicates, prepare = build_evidence(...)             # public perception, pixels or oracle
executor = SkillExecutor(environment, ModalVLAClient(), budget=settings.budget, ...)
controller = SkillOrchestrator(executor, planner, prepare=prepare, ...)
summary = controller.run()
```

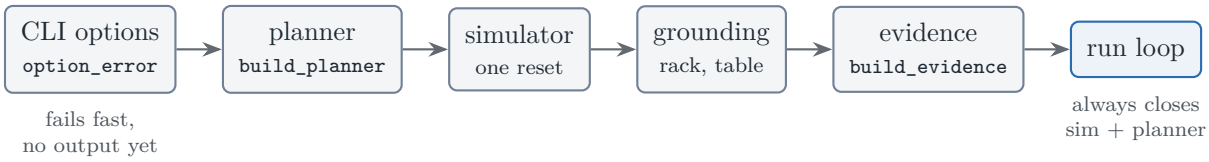


Figure 19: Episode start-up order.

11.2 Choosing the coach

Listing 4: `harness/runtime/assembly.py:92`

```
92  def build_planner(kind, run_settings, *, planner_settings=None, pixel_grounding=False):
93      """Return the planner for ``kind`` ("anthropic" or "scripted")."""
94      if kind == "anthropic":
95          return AnthropicPlanner(planner_settings, require_pixel_grounding=pixel_grounding)
96      if kind != "scripted":
97          raise ValueError("unsupported_planner")
98      if run_settings.handoff is not None:
99          return HandoffPlanner(run_settings.recipe, run_settings.handoff)
100     return GraspRetryPlanner(run_settings.recipe)
```

Where to change it

New planner? Write a class with `select(context) -> PlannerSelection` (protocol in `harness/planner/base.py`) and add one branch here. The loop never needs to know which planner it talks to.

11.3 What a move card looks like

Listing 5: harness/skills/schemas.py:84 (abridged)

```
class SkillCommand(Model):
    action: SkillType                                  # one of the six skills
    arm: Literal["left", "right", "both", "auto"] = "auto"
    parameters: VLAParameters | MoveParameters | ...   # typed per skill
    stop: StrictStr | None = None                      # a predicate name, for vla_act
    reason: Annotated[StrictStr, Field(max_length=1000)] | None = None
```

Every card is validated twice: once when Opus answers (`response_parser.py`) and again inside the executor. Unknown fields are rejected (`extra="forbid"`).

11.4 The heart of the turn loop

Listing 6: harness/runtime/orchestrator.py:259 (abridged)

```
selection = self.planner.select(PlannerContext(turn, observation.copy(), predicates, ...))
command = SkillCommand.model_validate(selection.command.model_dump())
self.log({**base, "event_type": "planner_selection", ...})   # intent logged before motion

prepared, rejection_code = self._prepare_anchor(command, observation, camera_to_avoid)
if rejection_code is not None:          # bad grounding: give feedback, try again
    grounding_feedback, grounding_camera_to_avoid = reselection_feedback(command, n)
    continue

previous = executor.execute(command)    # the only place robot motion starts
failure = turn_failure(command, previous, post)
flags = recovery_flags(previous)        # known, recoverable failure types
```

11.5 Playing a policy chunk

Listing 7: harness/skills/vla_act.py:62 (abridged)

```
while context.chunks < (success_chunk_limit or configured_chunk_limit):
    observation = env.observe()
    context.chunks += 1                          # failed requests also count
    actions = np.asarray(context.vla.infer(observation, parameters.chunk_size))
    # ... validate the whole chunk (shape 14, finite) before ANY motion ...
    for action in actions:
        stopped = predicate_stop()               # e.g. mug_lifted reached?
        if stopped:
            return stopped                       # remaining actions are discarded
        context.observation = env.step(action)
        if overlap_check():                      # the two arms touched
            return "arm_overlap_detected"
```

12 Cookbook: I want to...

I want to...	Do this
change what Opus is told	Edit `harness/prompts/common.md` or `full_task.md`. The prompt hash in `manifest.json` changes, so old and new runs stay distinguishable.
change budgets or chunk sizes	Edit the YAML in `configs/` (`budget:`, `anthropic.vla:`). No code change.
get feedback after more chunks	`anthropic.planner_feedback_chunks` (1–4) in the config.
use another segmentation model	Write a factory returning an object with `detect(images, *, observation_sequence)` and pass `--perception-factory pkg.module:factory`.
add a new planner	Implement `select`, register it in `assembly.build_planner`, add a CLI choice in `run_skills.build_parser`.
use another policy backend	Implement `reset_history()` and `infer(observation, n)` (`harness/vla/base.py`).
add a checklist question	Add the name to `PREDICATE_NAMES` (`predicates/registry.py`) and teach a provider to answer it; return `null` when unsure.
tune lift or mask thresholds	See the green box in Section 8.
add a new skill	Rare and risky: add a `SkillType`, parameter model, implementation, registry entry, tool description, and tests. Talk to the team first.
ask Opus a new perception question	Build it with `harness/perception/vision_request.py` and add a prompt file.

Careful

Do not send simulator-only facts to Opus, do not remove the release check, and do not let a failed command be retried automatically. These rules are what make results trustworthy.

13 Configuration files

A config is a YAML file read into `RunSettings` (</> `harness/runtime/settings.py`). This is the richest preset, `configs/skill_runtime_opus_full_task_rgbd_table_lift.yaml`:

Listing 8: An annotated full-task configuration

```yaml

record_video: true              # writes episode.mp4 into the run folder
max_turns: 100                  # planner turns
perception:
  rgbd_enabled: true            # capture depth + calibration
  opus_rack_grounding: true     # ground the rack once at reset
  opus_mug_lift_grounding: true # table-plane lift evidence
  rack_grounding_camera: right_camera
  rack_prompt_profile: explore_mug_rack
budget:
  max_skill_calls: 100
  max_analytic_ticks: 2500
  max_policy_actions: 800
  max_vla_chunks: 100
anthropic:
  model: claude-opus-5          # explicit; there is no fallback model
  task_scope: full_task         # grasp -> handoff -> hang
  planner_feedback_chunks: 1    # Opus looks again after every chunk
  vla: {max_chunks: 32, chunk_size: 8, max_policy_actions: 768}
```

Config family	Use
`skill_runtime.yaml`	small scripted acquisition (no Opus); good first run
`skill_runtime_rgbd.yaml`	same, with depth for a detector
`skill_runtime_left_transfer.yaml`, `skill_runtime_full_handoff.yaml`	scripted handoff experiments
`skill_runtime_opus_*.yaml`	Opus planner, from acquisition to full task
`explore_*.yaml`	campaigns for `run_exploration.py`
`evaluation.yaml`	frozen evaluation seeds for `run_evaluation.py`
`harness.yaml`, `baseline.yaml`	legacy pass-through hook for the baseline

14 Setting up and running

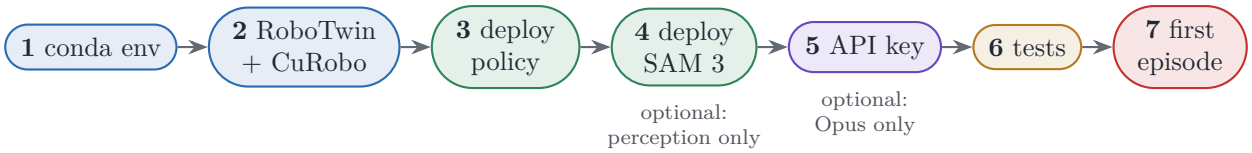


Figure 20: Deployment steps. You need Linux, Python 3.10 and an NVIDIA GPU with Vulkan.

Step 1–2: local environment and simulator

```
conda create -n robotwin python=3.10 -y && conda activate robotwin
python -m pip install -r requirements/sim.txt \
   -r requirements/planner.txt 'modal>=1,<2'
git clone https://github.com/RoboTwin-Platform/RoboTwin.git RoboTwin
git -C RoboTwin checkout ea8b21121ebb3cd201ff5b3fe361944ac94eda3f
git -C RoboTwin apply ../patches/robotwin-renderer.patch
(cd RoboTwin && bash scripts/_download_assets.sh)
git clone --branch v0.7.8 https://github.com/NVlabs/curobo.git \
   RoboTwin/envs/curobo
git -C RoboTwin/envs/curobo checkout d64c4b005459db10c5dd867d8b30a87d5bda9bdb
python -m pip install -e RoboTwin/envs/curobo --no-build-isolation
python -m pip install warp-lang==1.12.0
```

Keep NumPy at 1.26.4 locally; the cloud workers use their own Python 3.12 images.

Step 3–5: cloud workers and keys

```
modal setup
modal deploy modal_apps/policy.py   # frozen Neotix policy (T4)
modal deploy modal_apps/sam3.py     # needs Modal secret "huggingface" (HF_TOKEN)
export ANTHROPIC_API_KEY=...       # never commit it
```

Step 6: tests (CPU only, about 6 seconds)

```
python -m unittest discover -s tests -q
```

Step 7: run something

```
# scripted planner + development oracle:  no Opus calls, cheapest live check
python run_skills.py --config configs/skill_runtime.yaml \
   --development-oracle --output data/runs/smoke_01

# full task with Opus and public RGB-D perception
python run_skills.py \
   --config configs/skill_runtime_opus_full_task_rgbd_table_lift.yaml \
   --planner anthropic \
   --perception-factory harness.perception.modal_sam3:create_detector \
   --output data/runs/full_task_01
```

Careful

Use a new `--output` folder each time (the code refuses to overwrite). Run only one simulator job at a time on small GPUs. Choose exactly one evidence mode: oracle *or* public perception.

14.1 What you get after a run

```
data/runs/full_task_01/
├─ manifest.json          settings, seed, prompt hash
├─ assessments.jsonl      every planner choice and turn result
├─ skill_events.jsonl     every action, tick and check
├─ simulator_timing.jsonl how long RoboTwin took
├─ oracle.jsonl           private evidence (oracle mode only)
├─ observations/          saved pictures + depth (.npz)
├─ episode.mp4            video when record_video: true
├─ summary.json           outcome, counts, timings
└─ candidate_catalog.json memory ideas to review (never auto-used)
```

Figure 21: Run artifacts. Exit code 0 means a success outcome such as `acquisition_confirmed` or `task_success`.

14.2 Other ways to run

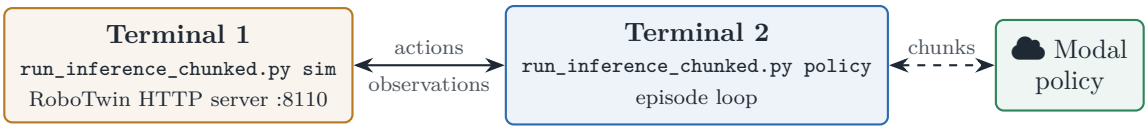


Figure 22: The baseline: the frozen policy alone, no coach. Used to compare against.

- **Exploration** (`run_exploration.py`) runs many episodes, each in a new process, and collects memory *candidates*. It currently requires `--development-oracle`.
- **Evaluation** (`run_evaluation.py freeze/run`) locks seeds, code hashes and memory, so results cannot be accidentally tuned on the test seeds.
- **Offline tools** live in `development/`, for example `python -m development.replays.perception --help`. Most need no simulator.

15 Where things stand today

Part	State	Notes
Baseline policy runner	✔ works	independent of the harness
Skill executor + six skills	✔ works	bounded, tested on CPU fakes
Scripted acquisition / handoff	✔ works	development-oracle evidence
Opus planner, full task	? exploring	phases succeed sometimes; not yet reliable hanging
Public RGB-D perception	? exploring	SAM 3 + Opus grounding; gaps return `null`
Evaluation with perception	✘ not yet	needs a frozen detector/prompt contract
Unit tests	✔ 425 pass	`python -m unittest discover -s tests`

Careful

- Success of one phase is **not** proof of reliable full-task hanging.
- RoboTwin's native success check is not the same as a verified handle-on-peg hang.
- Configs name the model `claude-opus-5`; check it is the model you intend to use.
- Checkpoints and provenance files recorded before the October 2026 restructure no longer match the current file layout; record new ones.

16 Testing and contributing

- **All tests run on CPU** with fake simulators, fake policies and fake Opus clients. No GPU or API key is needed.
- **Change behaviour, change a test.** New behaviour needs a test that shows it.
- **One subsystem per pull request**, with the contract change and a CPU or replay check.
- **Prompts are code.** Version a prompt change together with the parser/schema it serves.
- **Clean-code defaults** for agents and people are in `.agents/skills/clean-code/SKILL.md`.

A Frequently asked questions

Why not let Opus control the joints directly?

Language models are not precise with numbers and cannot react within milliseconds. Opus judges the situation; deterministic code and the trained policy do the motion.

Why does an unknown answer stop the robot from opening its hand?

Opening a hand that holds an unsupported mug drops it. Without proof, the safe choice is not to open.

What is the "development oracle" for?

Debugging. It uses simulator-only knowledge so we can test skills before perception is good enough. It is never shown to Opus and is not allowed in evaluation with perception.

Why is the policy called in short chunks?

So the coach can look again often. Long chunks are cheaper but react slower.

Where do I start reading code?

`run_skills.py`, then `harness/runtime/assembly.py`, then `harness/runtime/orchestrator.py`.